

XiaoMu Robot Programming Edition
API Documentation and
Programming Guide 1.0

Table of Contents

Chapter 1: Environment Preparation.....	4
1.1 Overview.....	4
1.2 Hardware Preparation.....	4
1.3 Connecting to Wi-Fi.....	4
1.4 Obtaining the Robot's IP Address.....	4
1.5 Remote Access to the Robot.....	5
1.5.1 Login Credentials.....	5
1.5.2 Logging in with SSH.....	5
1.5.3 Transferring Files with SFTP.....	6
1.6 Programming Environment Description.....	6
1.6.1 Python Version.....	7
1.6.2 ROS 2 Environment.....	7
1.6.3 Workspace Structure.....	7
1.7 Programming Examples Description.....	8
1.8 Building and Running.....	9
1.8.1 Building the Workspace.....	9
1.8.2 Sourcing the Environment.....	9
1.8.3 Running Example Programs.....	9
1.9 Creating Custom Programs.....	10
1.9.1 Adding a New Python File.....	10
1.9.2 Adding a New Launch Script.....	11
1.9.3 Modifying setup.py to Add Entry Point.....	11
1.9.4 Modifying package.xml to Add Dependencies.....	12
1.10 Important Notes.....	12
1.10.1 Program Conflict Handling.....	13
1.10.2 System Environment Notes.....	13
1.10.3 System Maintenance Reminders.....	13
1.11 Troubleshooting.....	13
1.12 Environment Setup Checklist.....	14
Chapter 2: ROS 2 Message Interface and API Reference.....	14
2.1 Overview.....	15
2.2 Message File Description.....	15
2.3 Sensor Messages (Subscribe).....	16
2.3.1 Temperature & Humidity Sensor (Sensor).....	16
2.3.2 IMU Sensor (Imu).....	16
2.3.3 Touch Sensor (Touch).....	17
2.3.4 Speech Recognition (Listen).....	17
2.3.5 RFID Reader (Rifd).....	错误!未定义书签。
2.3.6 Battery Status (BatteryStatus).....	18
2.3.7 Vision Recognition Result (Saw).....	18
2.4 Control Messages (Publish).....	19
2.4.1 Speech Output (Speak).....	19

2.4.2 Motion Control (Action).....	19
2.4.3 Vision Control (See).....	20
2.5 Message Interface Summary Table	20
2.6 API Cheat Sheet	21
Chapter 3: Integrated Application Development	21
3.1 Overview	21
3.2 Development Process.....	22
3.3 Integrated Application Examples.....	22
3.3.1 Speech Output (`xiaomu_speak.py`)	22
3.3.2 Touch Interaction (`xiaomu_touch.py`).....	22
3.3.3 Sensor Reading (`xiaomu_sense.py`).....	23
3.3.4 Motion Control (`xiaomu_move.py`)	23
3.3.5 Vision Recognition (`xiaomu_vision.py`).....	23
3.3.6 Intelligent Chat (`xiaomu_chat.py`).....	24
3.3.7 Basic Sensor Reading (`simple.py`)	24
3.4 Example Difficulty Gradient Description	24
3.5 Running Multiple Nodes Simultaneously	25
3.6 Adding Custom Programs.....	25
3.6.1 Adding a New Python File	25
3.6.2 Modifying setup.py to Add Entry Point.....	25
3.6.3 Creating a Launch File	25
3.6.4 Building and Running	25
3.7 Development Best Practices.....	26
3.8 Common Issues.....	26
3.9 Chapter Summary.....	26
Appendix A: Common Commands Quick Reference	27
Appendix B: Directory Structure	27
Appendix C: Copyright and Confidentiality Statement.....	28
C.1 Copyright Notice	28
C.2 Confidentiality Statement.....	28
C.3 Consequences of Violation	29

Chapter 1: Environment Preparation

1.1 Overview

XiaoMu Robot Programming Edition is equipped with the Linux operating system. Users can access the robot remotely via the network and conduct secondary development using Python 3.9 and the ROS 2 framework (IRON) . This chapter guides you through the preparation of the programming environment.

1.2 Hardware Preparation

Before you start programming, ensure the following devices are ready:

- XiaoMu Robot (powered on)
- Router (supports Wi-Fi)
- A computer connected to the network (Windows / macOS / Linux)

1.3 Connecting to Wi-Fi

XiaoMu Robot needs to connect to the local area network via Wi-Fi so that your computer can access it remotely.

Steps:

1. Start XiaoMu Robot by following the steps in the XiaoMu user manual.
2. Enter XiaoMu's network configuration mode (refer to the product manual for specific operations).
3. Select the Wi-Fi network you want to connect to and enter the password.
4. Wait for the robot to confirm a successful connection.

 ****Tip**:** When the connection is successful, the robot will announce a voice prompt like "Network connected".

1.4 Obtaining the Robot's IP Address

The IP address of XiaoMu Robot can be viewed through the router management interface.

Steps:

1. Enter the router's management address in your computer browser (usually `192.168.1.1` or `192.168.0.1`).
2. Log in with the router administrator username and password.
3. Find the menu such as "Connected Devices", "DHCP Client List" or "Device Management".
4. Look for a device named `linaro-alip` or similar in the list.
5. Record the IP address of this device (e.g. `192.168.1.101`).

 **Note:** The router management interface varies by brand and model. Please refer to the router manual for details.

1.5 Remote Access to the Robot

After obtaining the IP address, you can use SSH and SFTP tools to access the robot remotely.

1.5.1 Login Credentials

Item	Information
	----- -----
Username	`linaro`
Password	`linaro`
Port	`22` (default)

1.5.2 Logging in with SSH

For Windows users:

Recommended tools:

- **PuTTY** (lightweight SSH client)
- **MobaXterm** (more comprehensive functions)
- **PowerShell** (built-in for Windows 10/11)

Example with PowerShell:

```
```powershell
ssh linaro@<ROBOT_IP_ADDRESS>
```
```

For macOS / Linux users:

Open Terminal and run the following command:

```
``bash
ssh linaro@<ROBOT_IP_ADDRESS>
``
```

Example:

```
``bash
ssh linaro@192.168.1.100
``
```

When connecting for the first time, you will be prompted to confirm the fingerprint. Type `yes` to continue.

1.5.3 Transferring Files with SFTP

SFTP (SSH File Transfer Protocol) is used to transfer files between your computer and the robot.

****Recommended SFTP tools:****

| OS | Recommended Tools |
|---------|---------------------------------------|
| Windows | WinSCP, MobaXterm, FileZilla |
| macOS | Cyberduck, FileZilla, Terminal |
| Linux | Remmina, FileZilla, Terminal commands |

****SFTP via Terminal:****

```
``bash
sftp linaro@<ROBOT_IP_ADDRESS>
``
```

****Using GUI tools (WinSCP as example):****

1. Open WinSCP
2. Select `SFTP` as the file protocol
3. Enter the robot's IP address as the host name
4. Username `linaro`, password `linaro`
5. Click Login

1.6 Programming Environment Description

1.6.1 Python Version

XiaoMu Programming Edition supports **Python 3.9**:

```
``bash
# Verify Python version after SSH login
python3 --version
# Example output: Python 3.9.2
``
```

1.6.2 ROS 2 Environment

XiaoMu Robot runs based on the ROS 2 framework. Users need to understand basic ROS 2 concepts (nodes, topics, messages, etc.).

Common ROS 2 commands:

```
``bash
# List ROS 2 nodes
ros2 node list

# List topics
ros2 topic list

# Show message definition
ros2 interface show robot_interfaces/msg/Speak
``
```

1.6.3 Workspace Structure

The user workspace is located at `~/linaro/tm_study/`, with the following structure:

```
``bash
/home/linaro/tm_study/
├── src/                                # Source code directory
│   └── example/                        # Example package
│       └── example/                    # Python module directory
│           ├── __init__.py
│           ├── simple.py               # Example: Basic sensor reading
│           ├── xiaomu_speak.py         # Example: Audio output
│           ├── xiaomu_touch.py         # Example: Touch interaction
│           ├── xiaomu_sense.py         # Example: Sensor reading
│           └── xiaomu_move.py          # Example: Motion control
```

```

|           |   |—— xiaomu_chat.py      # Example: Intelligent chat
|           |   |—— xiaomu_vision.py    # Example: Vision recognition
|           |   |—— setup.py            # Package installation config
|           |   |—— package.xml         # Package dependency declaration
|—— launch/                                # Launch scripts directory
|   |—— simple_launch.py
|   |—— speak_launch.py
|   |—— touch_launch.py
|   |—— sense_launch.py
|   |—— move_launch.py
|   |—— chat_launch.py
|   |—— vision_launch.py
|—— install/                             # Compiled installation directory
...

```

****Directory / File Description:****

```

Directory / File	Purpose
`src/example/example/`	Python program files (*.py)
`launch/`	ROS 2 launch scripts (*.launch.py)
`src/example/setup.py`	Package installation configuration, defines entry points
`src/example/package.xml`	Package dependency declaration file
`install/`	Compiled output directory

```

1.7 Programming Examples Description

Complete programming examples are provided under the `tm\_study` directory to help you get started quickly:

```

Example File	Description
`simple.py`	Basic example: Subscribe to all topics including sensors, touch, IMU, vision, speech recognition, etc.
`xiaomu_speak.py`	Speech output example: Make XiaoMu speak
`xiaomu_touch.py`	Touch interaction example: Respond when head or belly is touched
`xiaomu_sense.py`	Sensor example: Read temperature, humidity, IMU data and announce
`xiaomu_move.py`	Motion control example: Nod, shake head, wave, etc.
`xiaomu_chat.py`	Intelligent chat example: Complete interaction integrating speech recognition, motion, touch
`xiaomu_vision.py`	Vision recognition example: Enable vision, recognize face emotions and objects and respond

```


****Supporting Launch Scripts (launch directory):****

- `simple\_launch.py`
- `speak\_launch.py`
- `touch\_launch.py`
- `sense\_launch.py`
- `move\_launch.py`
- `chat\_launch.py`
- `vision\_launch.py`

1.8 Building and Running

1.8.1 Building the Workspace

```
```bash
cd /home/linaro/tm_study
colcon build --packages-select example
```
```

Or build all modules directly:

```
```bash
cd /home/linaro/tm_study
colcon build
```
```

1.8.2 Sourcing the Environment

```
```bash
source /home/linaro/tm_study/install/setup.bash
```
```

1.8.3 Running Example Programs

****Unified launch method using ros2 launch:****

```

```bash
cd /home/linaro/tm_study

Basic example
ros2 launch launch/simple_launch.py

Speech output example
ros2 launch launch/speak_launch.py

Touch interaction example
ros2 launch launch/touch_launch.py

Sensor example
ros2 launch launch/sense_launch.py

Motion control example
ros2 launch launch/move_launch.py

Intelligent chat example
ros2 launch launch/chat_launch.py

Vision recognition example
ros2 launch launch/vision_launch.py
```

```

1.9 Creating Custom Programs

1.9.1 Adding a New Python File

User-created custom Python files should be placed in the following directory:

```

```bash
/home/linaro/tm_study/src/example/example/
```

```

****Example: Create a custom program****

```

```bash
Execute after SSH login
cd /home/linaro/tm_study/src/example/example/

```

```
touch my_custom_node.py
'''
```

You can also develop locally on your computer and transfer related files to the corresponding directory via SFTP.

## 1.9.2 Adding a New Launch Script

User-created launch scripts should be placed in the following directory:

```
'''bash
/home/linaro/tm_study/launch/
'''

Launch Script Template:
'''python
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
 return LaunchDescription([
 Node(
 package='example',
 namespace='tmmini',
 executable='my_custom_node',
 name='my_custom_node'
),
])
'''
```

## 1.9.3 Modifying setup.py to Add Entry Point

In `~/home/linaro/tm_study/src/example/setup.py`, find the `entry_points` section and add your program entry:

```
'''python
entry_points={
 'console_scripts': [
 'simple = example.simple:main',
 'speak = example.xiaomu_speak:main',
 'touch = example.xiaomu_touch:main',
 'sense = example.xiaomu_sense:main',
 'move = example.xiaomu_move:main',
],
}
```

```

 'chat = example.xiaomu_chat:main',
 'vision = example.xiaomu_vision:main',
 'my_custom_node = example.my_custom_node:main', # Add your program
],
},
...

```

**\*\*Format Description for entry\_points:\*\***

Part	Description
`executable_name`	Name used in the `executable` field of the launch file
`module_path`	Python module path relative to `/home/linaro/tm_study/src/example/`
`function_name`	The `main` function name in the Python program

## 1.9.4 Modifying package.xml to Add Dependencies

In `/home/linaro/tm\_study/src/example/package.xml`, add third-party Python dependencies:

```

<?xml
<!-- Add Python dependencies -->
<exec_depend>python3-requests</exec_depend>
<exec_depend>python3-numpy</exec_depend>
<exec_depend>python3-opencv</exec_depend>
...

```

**\*\*Common Dependencies:\*\***

Dependency	Purpose
`python3-numpy`	Numerical calculation
`python3-opencv`	Image processing
`python3-requests`	HTTP requests

 **\*\*Tip\*\*:** After adding dependencies, you need to rebuild the workspace for changes to take effect.

---

## 1.10 Important Notes

### 1.10.1 Program Conflict Handling

When users write their own programs, resource conflicts may occur with the robot's main control program (e.g., occupying the same sensors or actuators), causing abnormal operation of the main control.

**Solutions:**

1. **Stop the custom program:** Press `Ctrl + C` in the terminal to terminate the running user program.
2. **Restart the robot main control:** Reboot the robot to restore normal operation.

 **Recommendation:** During development and debugging, avoid occupying key resources required by the main control in your own programs.

### 1.10.2 System Environment Notes

If you are not familiar with the Linux system, please follow these rules:

-  **Do not modify system parameters** at will
-  **Do not modify system environment variables**
-  **Do not delete or modify key system directories**
-  **Only operate in your own working directory** (`/home/linaro/tm\_study/`)
-  **Only use SSH/SFTP for development and file transfer**

### 1.10.3 System Maintenance Reminders

- **Do not attempt to attack the robot system:** Including but not limited to cracking system permissions, maliciously modifying system files, etc.
- **Do not delete key system files.**
- **Maintain a good development environment together.** Thank you for your cooperation and understanding.

## 1.11 Troubleshooting

Issue	Possible Cause	Solution
-------	----------------	----------

SSH connection failed	Wrong IP or network unreachable	Reconfirm IP via router; check if robot and computer are on the same network
-----------------------	---------------------------------	------------------------------------------------------------------------------

Incorrect password	Caps lock or typo	Confirm password is `linaro` (lowercase); check keyboard
--------------------	-------------------	----------------------------------------------------------

SFTP transfer failed	Insufficient disk space	Run `df -h` to check space; clean unnecessary
----------------------	-------------------------	-----------------------------------------------

files |

| Program runtime error | Missing Python dependencies | Add dependencies in `package.xml` and rebuild |

| Build failed | Wrong `setup.py` configuration | Check if `entry\_points` format is correct |

| Launch file not found | Launch file misplaced or unconfigured | Confirm launch file is in `/home/linaro/tm\_study/launch/` |

| Conflict with main control | Resource occupation conflict | Stop custom program or reboot robot |

| Cannot find robot IP | Router does not show device | Confirm robot is connected to Wi-Fi; refresh router page |

---

## 1.12 Environment Setup Checklist

After completing this chapter, confirm the following items:

- [ ] Robot powered on and connected to Wi-Fi
- [ ] Obtained robot IP via router management page
- [ ] Successfully logged into robot via SSH (username/password: linaro/linaro)
- [ ] Able to transfer files to robot via SFTP
- [ ] Confirmed Python 3.9 environment is normal
- [ ] Understood basic ROS 2 concepts
- [ ] Understood workspace structure (`/home/linaro/tm\_study/`)
- [ ] Understood Python file location (`src/example/example/`)
- [ ] Understood Launch file location (`/home/linaro/tm\_study/launch/`)
- [ ] Understood how to add custom program entry (`setup.py`)
- [ ] Understood how to add third-party dependencies (`package.xml`)
- [ ] Understood how to build and run programs
- [ ] Familiar with provided programming examples

---

## Chapter 2: ROS 2 Message Interface and API Reference

## 2.1 Overview

All functional modules of XiaoMu Robot communicate through ROS 2 message interfaces. Users can obtain sensor data, control robot motion, output speech, etc., by subscribing and publishing specific topics.

This chapter introduces the message interfaces provided by XiaoMu Robot. \*\*For detailed definitions of messages (including field types, constant values, etc.), please refer to the results queried by the `ros2 interface show` command.\*\*

```
> 💡 **How to query message definitions**:
> ```bash
> # List all available messages
> ros2 interface list | grep robot_interfaces
>
> # Show specific message definition
> ros2 interface show robot_interfaces/msg/Speak
> ros2 interface show robot_interfaces/msg/Touch
> ros2 interface show robot_interfaces/msg/Action
> ```
```

---

## 2.2 Message File Description

XiaoMu Robot uses the custom message package `robot\_interfaces`, and you need to import the corresponding message types in Python scripts:

```
```python  
from robot_interfaces.msg import Sensor, Imu, Touch, Speak, Listen, Action, Servo, See, Saw,  
Emotion, Led, Rfid, BatteryStatus  
```
```

**\*\*Message Package Dependency\*\***: Add the following in `package.xml`:

```
```xml  
<depend>robot_interfaces</depend>  
```
```

---

## 2.3 Sensor Messages (Subscribe)

The following messages are used to obtain sensor data by subscribing to corresponding topics.

| Module                 | Topic Name            | Message Type               | Description                       |
|------------------------|-----------------------|----------------------------|-----------------------------------|
| Temperature & Humidity | <code>/sensor</code>  | <code>Sensor</code>        | Get temperature and humidity data |
| IMU                    | <code>/imu</code>     | <code>Imu</code>           | Get attitude and motion data      |
| Touch                  | <code>/touch</code>   | <code>Touch</code>         | Get touch status                  |
| Speech Recognition     | <code>/listen</code>  | <code>Listen</code>        | Get speech recognition text       |
| RFID                   | <code>/rfid</code>    | <code>Rfid</code>          | Get RFID card UID                 |
| Battery                | <code>/battery</code> | <code>BatteryStatus</code> | Get battery status                |
| Vision Result          | <code>/saw</code>     | <code>Saw</code>           | Get vision recognition results    |

### 2.3.1 Temperature & Humidity Sensor (Sensor)

**Topic:** `/sensor`

**Usage Example:**

```
python
def sensor_callback(self, msg):
 temp = msg.temperature
 humidity = msg.humidity
 self.get_logger().info(f'Temperature: {temp:.1f}°C, Humidity: {humidity:.1f}%')

 if temp > 35:
 self._speak("It's so hot, please cool me down")
...

```

### 2.3.2 IMU Sensor (Imu)

**Topic:** `/imu`

**Usage Example:**

```
python
import math

def imu_callback(self, msg):
 ax, ay, az = msg.accel_x, msg.accel_y, msg.accel_z
```



```

 pitch = math.atan2(ax, az) * 180 / math.pi
 self.get_logger().info(f'Pitch angle: {pitch:.1f}°')
 ...

```

### 2.3.3 Touch Sensor (Touch)

**\*\*Topic\*\*:** `/touch``

**\*\*Description\*\*:** A value of 0 for `head_touch_state`` and `belly_touch_state`` means no touch, 1 means touched.

**\*\*Usage Example\*\*:**

```

``python
def touch_callback(self, msg):
 if msg.head_touch_state == 1:
 self._speak("Hehe, patting my head feels so good")
 self._nod()

 if msg.belly_touch_state == 1:
 self._speak("Hee hee, that tickles")
 self._shake_head()
 ...

```

### 2.3.4 Speech Recognition (Listen)

**\*\*Topic\*\*:** `/listen``

**\*\*Usage Example\*\*:**

```

``python
def listen_callback(self, msg):
 text = msg.data.lower()

 if 'hello' in text:
 self._speak("Hello! Nice to see you")
 self._wave()
 elif 'goodbye' in text:
 self._speak("Goodbye, see you next time")
 self._wave()
 ...

```

---

### 2.3.5 Battery Status (BatteryStatus)

**\*\*Topic\*\*:** `/battery``

**\*\*Usage Example\*\*:**

```
```python
def battery_callback(self, msg):
    if msg.status == 1:
        self.get_logger().info('Charging')
    elif msg.status == 2:
        self.get_logger().info('Battery full')
    else:
        self.get_logger().info(f'Current battery: {msg.percentage}%%')
...
```
```

---

### 2.3.6 Vision Recognition Result (Saw)

**\*\*Topic\*\*:** `/saw``

**\*\*Description\*\*:** The message contains two lists, `persons`` and `things``, storing detected people and objects respectively.

**\*\*Usage Example\*\*:**

```
```python
def saw_callback(self, saw_data):
    for person in saw_data.persons:
        if person.emotion == 1:
            self._wave()
            self._speak("You look happy!")

    for thing in saw_data.things:
        if thing.type == "cat":
            self._speak("What a cute cat")
...
```
```

---

## 2.4 Control Messages (Publish)

The following messages are used to control the robot by publishing to corresponding topics.

| Module         | Topic Name            | Message Type          | Description                   |
|----------------|-----------------------|-----------------------|-------------------------------|
| Speech Output  | <code>/speak`</code>  | <code>`Speak`</code>  | Broadcast text                |
| Motion Control | <code>/action`</code> | <code>`Action`</code> | Control servos and LEDs       |
| Vision Control | <code>/eye`</code>    | <code>`See`</code>    | Start/Stop vision recognition |

### 2.4.1 Speech Output (Speak)

**Topic:** `/speak``

**Usage Example:**

```
python
def _speak(self, text):
 msg = Speak()
 msg.text = text
 self.speak_pub.publish(msg)
```

Note: The output text must match the robot's current system language, otherwise abnormal broadcast will occur.

---

### 2.4.2 Motion Control (Action)

**Topic:** `/action``

**Description:**

- ``Action`` message includes `time`` (execution time), `type`` (action type), `servos`` (servo list), `leds`` (LED list)
- For action type definitions, check constants via `ros2 interface show``

**Usage Example:**

```
python
def _wave(self):
 action = Action()
 action.time = 0.5
 action.type = 0
```

```

 servo = Servo()
 servo.index = 4
 servo.angle = 45
 action.servos.append(servo)

 self.action_pub.publish(action)
 ...

 """python
def reset(self, duration=1.0):
 action = Action()
 action.time = float(duration)
 action.type = 2 # Reset type
 self.action_pub.publish(action)
 ...

```

### 2.4.3 Vision Control (See)

```

Topic: `eye`

Usage Example:
 """python
def start_vision(self):
 see = See()
 see.command = 1
 self.eye_pub.publish(see)
 self._speak("Vision recognition enabled")
 ...

```

## 2.5 Message Interface Summary Table

| Module                 | Topic Name | Message Type | Direction | Description                       |
|------------------------|------------|--------------|-----------|-----------------------------------|
| Temperature & Humidity | `/sensor`  | `Sensor`     | Subscribe | Get temperature and humidity data |
| IMU                    | `/imu`     | `Imu`        | Subscribe | Get attitude and motion data      |
| Touch                  | `/touch`   | `Touch`      | Subscribe | Get touch status (0/1)            |
| Speech Recognition     | `/listen`  | `Listen`     | Subscribe | Get recognition text              |

```
Battery	`/battery`	`BatteryStatus`	Subscribe	Get battery status
Vision Result	`/saw`	`Saw`	Subscribe	Get vision recognition results
Speech Output	`/speak`	`Speak`	Publish	Broadcast text
Motion Control	`/action`	`Action`	Publish	Control servos and LEDs
Vision Control	`/eye`	`See`	Publish	Start/Stop vision
```

---

## 2.6 API Cheat Sheet

```
Function	Operation	Interface
Make XiaoMu speak	Publish	`Speak`
Control servos	Publish	`Action` + `Servo`
Reset servos	Publish	`Action` (Reset type)
Start vision	Publish	`See` (start command)
Read temperature & humidity	Subscribe	`Sensor`
Read attitude	Subscribe	`Imu`
Read touch	Subscribe	`Touch`
Read speech	Subscribe	`Listen`
Get vision results	Subscribe	`Saw`
```

> 💡 **\*\*Important Note\*\***: Field types, constant values, servo indexes, etc., in messages are subject to the actual output of the `ros2 interface show` command.

---

# Chapter 3: Integrated Application Development

## 3.1 Overview

After mastering the message interfaces in Chapter 2, this chapter guides you to comprehensively use each module to develop robot applications with practical functions. By combining speech, motion, sensors, vision and other capabilities, you can create rich interactive experiences.

---

## 3.2 Development Process

Developing a complete robot application usually follows these steps:

1. **Define functional requirements**: Clarify the scenarios and behaviors the robot needs to respond to
2. **Design interaction logic**: Plan trigger conditions, response actions, state transitions
3. **Write node code**: Implement subscription and publishing logic
4. **Add launch scripts**: Create launch files for easy startup
5. **Build and test**: Compile workspace and run verification

---

## 3.3 Integrated Application Examples

The following examples are based on existing files in the ``tm_study/src/example/example/`` directory, arranged from simple to complex. You can study and refer according to your needs.

### 3.3.1 Speech Output (``xiaomu_speak.py``)

**Function**: Basic example of making XiaoMu speak, demonstrating the most basic speech output capability.

**Interfaces involved**: ``Speak`` (speech output)

**Launch Command**:

```
``bash
ros2 launch launch/speak_launch.py
``
```

---

### 3.3.2 Touch Interaction (``xiaomu_touch.py``)

**Function**: XiaoMu responds with speech and motion when head or belly is touched.

**Interfaces involved**: ``Touch`` (touch detection), ``Speak`` (speech output), ``Action`` (motion control)

```
Launch Command:
```bash  
ros2 launch launch/touch_launch.py  
```  

```

### 3.3.3 Sensor Reading (`xiaomu\_sense.py`)

**\*\*Function\*\*:** Read temperature, humidity and IMU sensor data and announce.

**\*\*Interfaces involved\*\*:** `Sensor` (temperature & humidity), `Imu` (attitude sensor), `Speak` (speech output)

```
Launch Command:
```bash  
ros2 launch launch/sense_launch.py  
```  

```

### 3.3.4 Motion Control (`xiaomu\_move.py`)

**\*\*Function\*\*:** Control XiaoMu to perform various actions, including nod, shake head, wave, reset, etc.

**\*\*Interfaces involved\*\*:** `Action` (motion control), `Speak` (speech feedback)

```
Launch Command:
```bash  
ros2 launch launch/move_launch.py  
```  

```

### 3.3.5 Vision Recognition (`xiaomu\_vision.py`)

**\*\*Function\*\*:** Enable vision recognition, recognize face emotions and objects and respond.

**\*\*Interfaces involved\*\*:** `See` (vision control), `Saw` (vision result), `Speak` (speech output), `Action` (motion control)

```

Launch Command:
```bash
ros2 launch launch/vision_launch.py
```

```

### 3.3.6 Intelligent Chat (`xiaomu\_chat.py`)

**\*\*Function\*\*:** Complete interactive example integrating speech recognition, motion control, touch interaction.

**\*\*Interfaces involved\*\*:** `Listen` (speech recognition), `Touch` (touch detection), `Speak` (speech output), `Action` (motion control)

```

Launch Command:
```bash
ros2 launch launch/chat_launch.py
```

```

### 3.3.7 Basic Sensor Reading (`simple.py`)

**\*\*Function\*\*:** Subscribe to all sensor topics, demonstrate data reading methods, suitable as a reference template for developing custom nodes.

**\*\*Interfaces involved\*\*:** `Sensor`, `Imu`, `Touch`, `Listen`, `Saw`, `Rfid`, `BatteryStatus`

```

Launch Command:
```bash
ros2 launch launch/simple_launch.py
```

```

## 3.4 Example Difficulty Gradient Description

| Difficulty   Example   Core Learning Objective              |
|-------------------------------------------------------------|
| ----- ----- -----                                           |
| ★☆☆   `xiaomu_speak.py`   Master message publishing (Speak) |



| ★☆☆ | `xiaomu\_touch.py` | Master message subscription (Touch) + simple response |  
| ★★☆☆ | `xiaomu\_sense.py` | Master sensor data processing |  
| ★★☆☆ | `xiaomu\_move.py` | Master motion control (Action + Servo) |  
| ★★★★★ | `xiaomu\_vision.py` | Master vision module control and result processing |  
| ★★★★★ | `xiaomu\_chat.py` | Comprehensive use of multiple modules for complete interaction |  
|  
| ★★★★★ | `simple.py` | Understand all message interfaces, use as development template |

---

## 3.5 Running Multiple Nodes Simultaneously

You can launch multiple nodes at the same time using one launch file, referring to the format of existing files in the `launch` directory.

---

## 3.6 Adding Custom Programs

### 3.6.1 Adding a New Python File

Create a new `.py` file in the `/home/linaro/tm\_study/src/example/example/` directory, referring to the structure of existing examples.

### 3.6.2 Modifying setup.py to Add Entry Point

Add the entry of the new program in `entry\_points` of `setup.py`:

```
```python
'my_node = example.my_node:main',
```
```

### 3.6.3 Creating a Launch File

Create the corresponding `\_launch.py` file in the `/home/linaro/tm\_study/launch/` directory.

### 3.6.4 Building and Running

```
```bash
```

```
cd /home/linaro/tm_study
colcon build --packages-select example
source install/setup.bash
ros2 launch launch/my_node_launch.py
'''
```

3.7 Development Best Practices

Practice	Description
Single Responsibility	Each node is responsible for only one clear functional module
Loose Coupling	Nodes communicate via topics, avoid direct dependencies
Parameterization	Use <code>declare_parameter</code> and <code>get_parameter</code> to make nodes configurable
Log Level	Use <code>debug/info/warn/error</code> levels for logging
Exception Handling	Catch exceptions to avoid node crash

3.8 Common Issues

Issue	Solution
No response after node startup	Run <code>source install/setup.bash</code> to load environment
No topic data received	Check if <code>namespace</code> in launch file is set to <code>tmmini</code>
Message not found during build	Confirm <code><depend>robot_interfaces</depend></code> exists in <code>package.xml</code>
Multiple node conflicts	Stop other nodes or use different namespaces

3.9 Chapter Summary

The `tm_study` workspace provides programming examples from simple to complex, covering speech, motion, sensors, touch, vision and other modules. It is recommended to learn in the following path:

1. Start with `xiaomu_speak.py` to understand the basic publish-subscribe model
2. Learn touch, sensor, motion control examples in sequence

3. Try comprehensive examples such as vision recognition and intelligent chat
4. Refer to `simple.py` to understand all interfaces and start developing your own nodes

The launch scripts corresponding to each example are located in the `/home/linaro/tm\_study/launch/` directory, one-to-one with the example files.

Appendix A: Common Commands Quick Reference

```
``bash
# ROS 2 environment setup (executed by default when SSH login)
source /opt/ros2/iron/setup.bash

# Project environment setup
source /home/linaro/tm_study/install/setup.bash

# Build
cd /home/linaro/tm_study && colcon build --packages-select example

# List topics
ros2 topic list

# Show message definition
ros2 interface show robot_interfaces/msg/MESSAGE_NAME

# Launch example
ros2 launch launch/EXAMPLE_NAME_launch.py
``
```

Appendix B: Directory Structure

```

```bash
/home/linaro/tm_study/
├── src/example/example/ # Python program directory
│ ├── simple.py
│ ├── xiaomu_speak.py
│ ├── xiaomu_touch.py
│ ├── xiaomu_sense.py
│ ├── xiaomu_move.py
│ ├── xiaomu_chat.py
│ └── xiaomu_vision.py
├── launch/ # Launch script directory
│ ├── simple_launch.py
│ ├── speak_launch.py
│ ├── touch_launch.py
│ ├── sense_launch.py
│ ├── move_launch.py
│ ├── chat_launch.py
│ └── vision_launch.py
└── install/ # Compiled output directory
```

```

Appendix C: Copyright and Confidentiality Statement

C.1 Copyright Notice

This document and all its contents (including but not limited to text, images, code examples, interface definitions, etc.) are the copyright property of **Timu Technology**.

No individual or organization may, without the prior written permission of Timu Technology:

- Copy, reproduce, or distribute all or part of this document
- Publish the contents of this document on the internet, forums, social media, or any other public channels
- Use this document for commercial purposes

C.2 Confidentiality Statement

This document contains technical information, communication interface definitions, and development methods related to Timu Technology products, which constitute **confidential information** of Timu Technology.

As a user of the XiaoMu Robot Programming Edition, you agree to:

1. **Do not distribute:** Do not provide this document or any technical information contained herein to any third party in any form
2. **Do not disclose:** Do not publish the contents of this document or any technical implementation based on it on the internet, GitHub, technical forums, or other public channels
3. **Internal use only:** This document is intended solely for your own or your team's internal use for secondary development of the XiaoMu robot

C.3 Consequences of Violation

Unauthorized dissemination or public disclosure of the contents of this document may constitute infringement. Timu Technology reserves the right to pursue legal remedies.

Thank you for your understanding and cooperation. For any questions, please contact Timu Technology official support.